Module 12 · SW Architecture

Layered Architecture

The classic n-tier structure — the default most systems start with.

Akın Kaldıroğlu · Lead Trainer

The idea

Organize by Technical Role

Layered architecture stacks the system into horizontal layers, each with one technical responsibility, each using the layer below it.

Simple

An organizing principle everyone already knows.

Ubiquitous

The default for most enterprise applications.

A baseline

Understand it well — then know when to move beyond it.

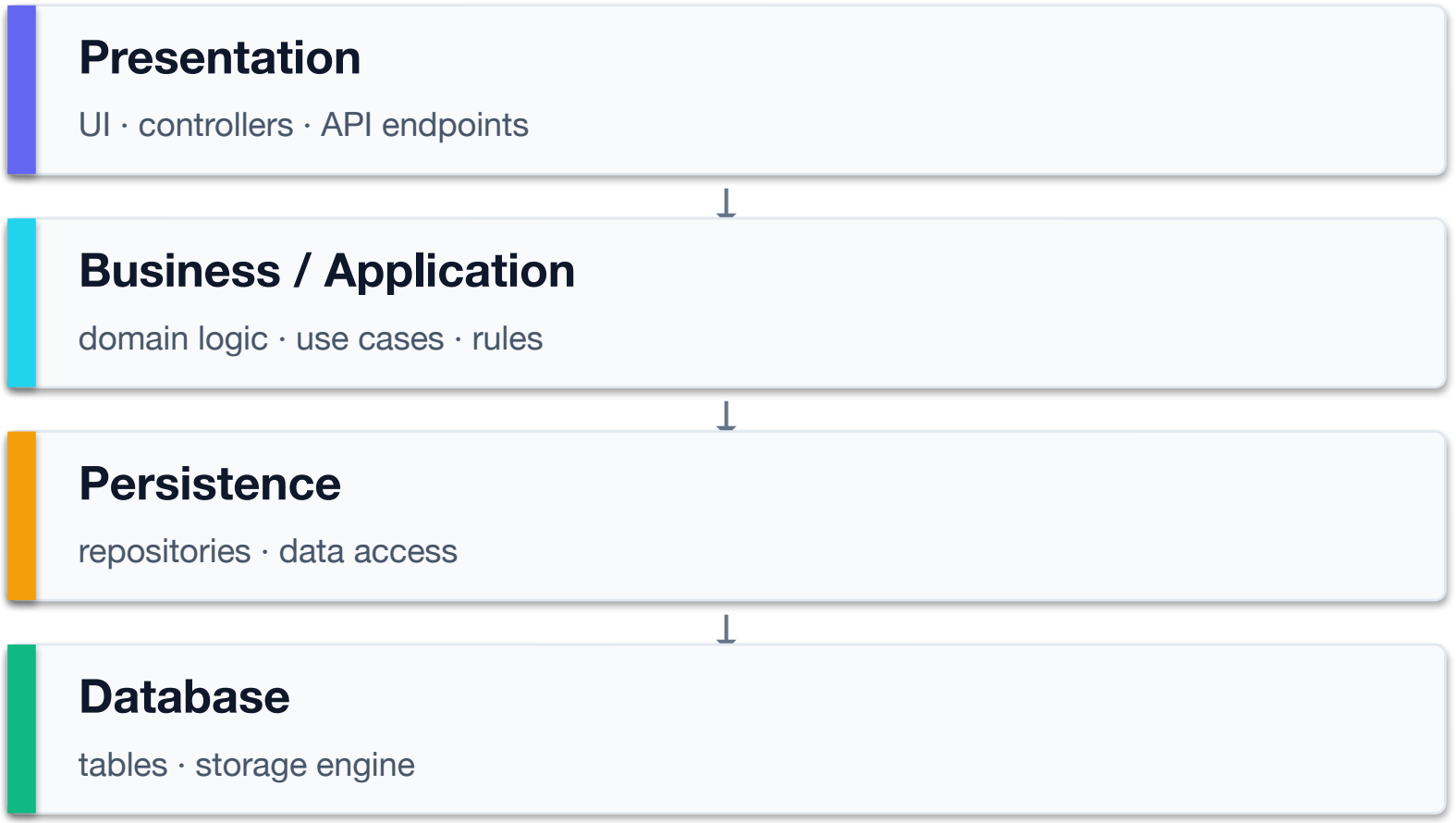
What you'll leave with

Learning Objectives

- Describe the classic layers and the rules between them.
- Trace a request as it flows down and back up the stack.
- Weigh the real benefits — and the real costs.
- Recognize the pain points that motivate hexagonal architecture.

The structure

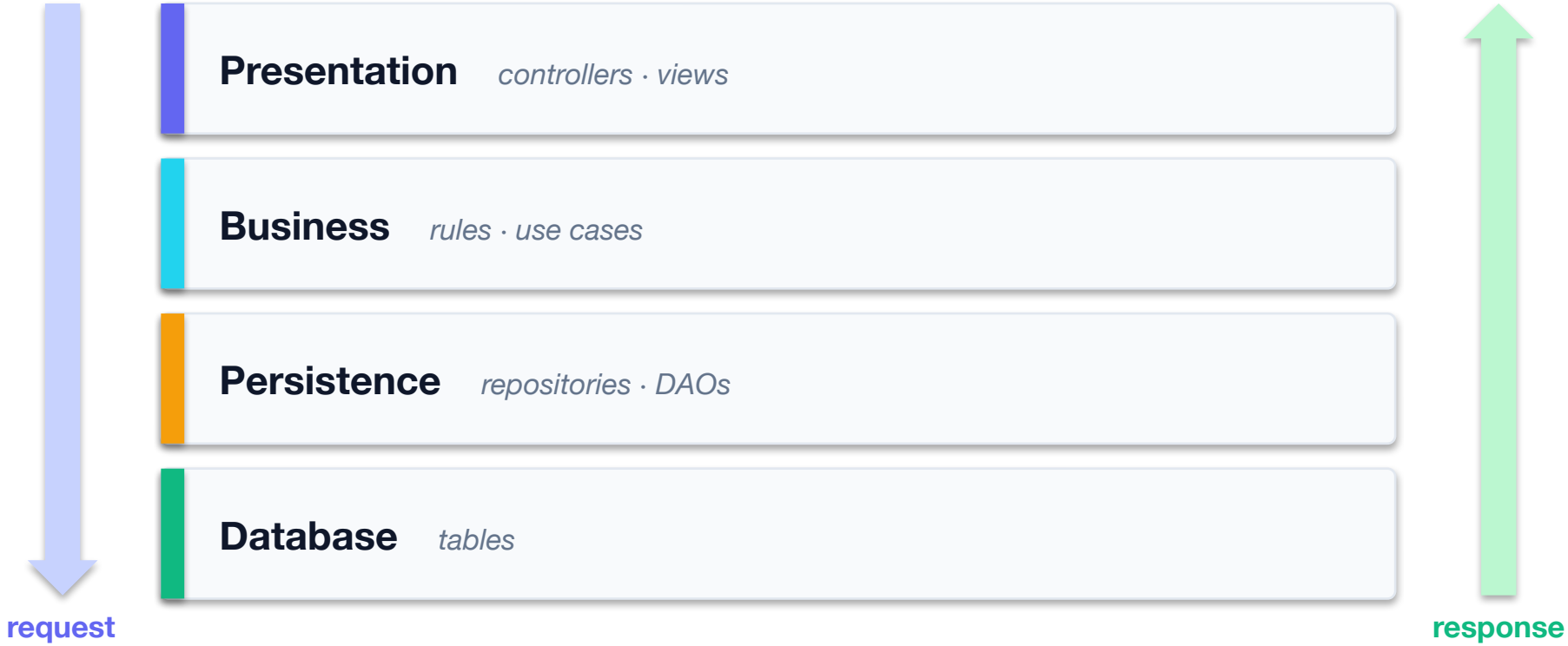
The Classic Layers



The drawing

The Picture Everyone Draws

A request enters at the top, falls through every layer, and comes back the same way. This is the shape the whole industry has in its head.



Every layer is closed: nothing may skip a level. That is the whole rule — and it says nothing about what a layer is allowed to know.

The one rule

Dependencies Point Downward

Each layer may use the layer directly below it — and nothing reaches upward. The presentation layer never talks straight to the database.

Allowed

- Presentation → Business
- Business → Persistence
- Persistence → Database

Not allowed

- Skipping a layer to save typing
- A lower layer calling upward
- Business logic in the UI or the DB

In practice

▶ runnable `dev.kaldiroglu.bootcamp.layered`

A Request, Layer by Layer

Flow *Down to persist, back up with a result*

```
POST /orders                          (Presentation)
  → OrderController.place(req)
     → OrderService.place(order)      (Business: rejects a blank order)
        → OrderRepository.save()      (Persistence)
           → INSERT INTO orders       (Database)
  ← 201 Created                       (back up the stack)
```

Each layer does its job and delegates the rest downward. Clean and familiar.

`presentation/OrderController` · `business/OrderService` · `persistence/OrderRepository`

`github.com/kaldiroglu/Agentic-Software-Design-and-Architecture-Bootcamp-`**`JAVA`** · **`-DOTNET`** · **`-PYTHON`**

The upside

Why It's Everywhere

Familiar

Every developer understands the structure instantly.

Separation

UI, logic and data access are clearly divided.

Simple start

Little ceremony — great for smaller systems.

The downside

Where Layered Hurts

The convenience hides a coupling problem: your business logic ends up depending on your infrastructure.

The pain

- Domain depends on the persistence layer
- Hard to test business logic without a DB
- Database-centric — schema drives design

The symptom

- Swap the DB? The domain feels it
- Slow, brittle tests through all layers
- Logic leaks up into the UI, down into SQL

The downside

Nothing Stops the Domain Reaching Out

Layered protects business code from being scattered — never from what it depends on:

- business rule
- steps outside, mid-rule
- plumbing

C#

the rule cannot finish without leaving its own layer

```
public User ChangePassword(string username, PasswordChangeRequest change)
{
    var currentUser = userRepository.FindByUsername(username)
        ?? throw new UsernameNotFoundException("Username not found.");
    var currentPassword = currentUser.Password;
    var newPassword = encoder.Encode(change.NewPassword);
    var tex = new TValidationException("password");
    if (!IsPasswordValid(change.NewPassword))
        tex.RejectNow("pattern", "not.valid");
    if (!encoder.Matches(change.OldPassword, currentPassword))
        tex.RejectNow("old", "not.matches");
    if (encoder.Matches(change.NewPassword, change.OldPassword))
        tex.RejectNow("old", "same");
    currentUser.Password = newPassword;
    return UpdateUser(currentUser);
}
```

Nothing here is illegal — that is the complaint. The layer below is always one call away, so no rule ever has to stay put.

An alternative

Evans Used Layers Too

Domain-Driven Design keeps the layers and renames them. Chapter 4 is called Isolating the Domain — the goal is stated, not assumed.

Classic n-tier	Evans · DDD, 2003	What changes
Presentation	User Interface	—
Business	Application	coordinates work; holds no rules
—	Domain	the model and the rules, alone
Persistence + DB	Infrastructure	all technology, one layer

Infrastructure is still the bottom layer, so the domain may still call down. What saves it is not the layering — the Repository interface belongs to the domain, and only the implementation is infrastructure.

“It explains the hexagonal architecture, which has emerged as a better description of what we do than the layered architecture.” — Eric Evans, foreword to Implementing Domain-Driven Design, 2013

Judgement

When Layered Is the Right Call

- Small to medium apps with straightforward domains.
- Teams that value familiarity and speed to first release.
- When the domain is simple enough that DB-coupling won't bite.
- As a starting point you can evolve — not a life sentence.

The turning point

So What's the Fix?

The core problem is direction: high-level policy depends on low-level detail. Recall DIP — what if we inverted that dependency at the architectural scale?

Put the domain at the center. Make infrastructure depend on it — not the other way round.

That idea is Hexagonal / Onion / Clean architecture — our next session.

Run it yourself

The Code Behind This Session

Every example in this session compiles and runs — the same lessons in three languages, with the smell and the fix sitting side by side:

Java

Maven · JUnit 5 · 2 tests here

`layered/presentation/`
OrderController — the way in

`layered/business/`
OrderService — the rules

`layered/persistence/`
OrderRepository — and who depends on whom

C# / .NET

xUnit · 2 tests here

`Architecture.cs`
layered and hexagonal, side by side

`ArchitectureTests.cs`
a request flows down and back

Python

pytest · 2 tests here

`architecture/layered/`
presentation · business · persistence

`tests/test_architecture.py`
the flow through the layers

`github.com/kaldiroglu/Agentic-Software-Design-and-Architecture-Bootcamp-JAVA · -DOTNET · -PYTHON`

Follow the import directions. In this package business still points down at persistence — that is the dependency hexagonal will invert.

Clone one, run the suite, then break something and watch which test goes red. ***Across all chapters the suites are 70 · 63 · 65 tests.***

Remember these

Key Takeaways

- ✓ Layered architecture stacks the system by technical role.
- ✓ The rule: dependencies point downward, never up or skipping.
- ✓ It's simple, familiar, and a fine default for many systems.
- ✓ Its weakness: the domain ends up coupled to infrastructure.
- ✓ That coupling — and DIP — is what motivates hexagonal architecture.

Go deeper

Further Reading

The books behind this session. If you read only one, read the first.

📖 **Patterns of Enterprise Application Architecture** Martin Fowler · Addison-Wesley, 2002

Layering, Service Layer, Repository and the Domain Model — the vocabulary of this style.

📖 **Pattern-Oriented Software Architecture, Vol. 1** Buschmann et al. · Wiley, 1996

The Layers pattern in its original, careful form — forces, benefits and liabilities.

📖 **Domain-Driven Design** Eric Evans · Addison-Wesley, 2003

Ch. 4 — the classic four-layer split and why the domain layer must stay isolated.

📖 **Software Architecture Patterns, 2nd ed.** Mark Richards · O'Reilly, 2022

A short, free survey that places layered architecture against the alternatives.

Next session

What's Next

Module 13

Hexagonal / Onion / Clean Architecture

with Akın Kaldıroğlu

Invert the dependencies: put the domain at the center and push databases, frameworks and UIs to the edges.

Reference implementation

Ayvalık Bank — Layered (LA)

A full banking system in the classic layered style. Follow a request down the layers, in three languages.

Ayvalık Bank · Layered Architecture

Java `github.com/kaldiroglu/AyvalikBankLA-JAVA`

.NET `github.com/kaldiroglu/AyvalikBankLA-NET`

Python `github.com/kaldiroglu/AyvalikBankLA-Python`

IN THIS SESSION, LOOK AT

- `web/controller -> service -> repository -> model`
- `note how service depends on repository (downward)`

Questions?

Know layered well — then know when to leave it.